Compilers
Spring 2026
Project Description

Project Timeline

Team Formation: Deadline: Feb 28th 2026.
Deadline to submit reports-source code-ppt files: Sunday May 3rd 2026.
Defenses: Wednesday May 6th 2026.

Project Overview

The goal of this project is to build a basic compiler for a custom programming language. The language will be simple yet functional, including basic arithmetic operations, variable assignments, and control structures like conditionals and loops.

You will use **Python** for this project, as it is a versatile language with excellent libraries for text processing, parsing, and building a compiler.

The project will be done in teams of 5 or 6 students.

Compiler Design Milestones:

Milestone 1: Lexical Analysis (Scanner)

In this milestone you will write a **lexical analyzer** (scanner) that converts the raw source code into a series of tokens. These tokens will represent language constructs such as numbers, variables, operators, and keywords.

Let's define a very basic language with the following syntax:

- **Variables:** x = 5; (assignment)
- **Arithmetic Operations:** x = x + 2;
- **Control Structure:** if x > 10 then { x = x - 1; }

1.1 Tasks:

- Implement a tokenizer that reads source code line by line.
- Identify the following token types:
  - **Keywords:** if, then, else
  - **Identifiers:** Variable names like x, y
  - **Operators:** +, -, =, >, {, }, ;
  - **Literals:** Integer values like 5, 10
- Use regular expressions or manual scanning techniques to identify the different token types.
- Output a stream of tokens that the syntax analyzer will use.

Examples:

Input	**Output**
x = 5 + 2;	[ID('x'), '=', NUM(5), '+', NUM(2), ';']
z = x * (y - 3);	[ID('z'), '=', ID('x'), '*', '(', ID('y'), '-', NUM(3), ')', ';']

if a = b then
  a = a + 1;
else
  b = b - 1;
	[KW('if'), ID('a'), '=', ID('b'), KW('then'), ID('a'), '=', ID('a'), '+', NUM(1), ';', KW('else'), ID('b'), '=', ID('b'), '-', NUM(1), ';']

1.2 Tools/libraries that you can use:

- Python's **re** module for regular expressions.
- Manual state machine design to handle tokenization.

1.3 Deliverable:

Lexer: Python code that reads source code and produces a token list.

Milestone 2: Syntax Analysis (Parser)

In this milestone you will build a **syntax analyzer** (parser). This component will take the tokens generated by the lexer and ensure they are arranged correctly according to the grammar of the language. It will create a **syntax tree** (AST - Abstract Syntax Tree).

2.1 Language Definition:

This language supports basic arithmetic operations, variable assignments, and simple data types.

Features:

1. **Variable Declarations:** A declaration defines a variable's type (either int or string) and its identifier. For example: int x; or string name;.
2. **Variable Assignments:** After declaration, a variable can be assigned a value in the form: variable = expression;. For example: x = 5; or name = "Alice";.
3. **Expressions:** Expressions can involve addition (+) and can include variables, numbers, or strings. Expressions can be added together using the + operator. For example, x + 2 is an expression, as is "hello" + " world!".
4. **Supported data types:** integers and strings.

2.2 Tasks:

You will build a syntax analyzer for the language defined in **2.1.**

- This is an example language grammar for basic arithmetic expressions with variable assignment and addition:

  <stmt> ::= <id> = <expr>;

  <expr> ::= <expr> + <term> | <term>

<term> ::= <id> | <num>

You are required to define the grammar **of the language defined in 2.1.**

- Implement a recursive descent parser or use a parser generator like PLY or Lark to construct its syntax tree.
- Handle errors when the code does not conform to the grammar (invalid syntax).

2.3 Example: For input x = 5 + 2;, the parser will output an AST that represents the statement.

2.4 Tools/libraries that you can use:

- **PLY** (Python Lex-Yacc) for parser generation.
- **Lark** for more flexible parsing.

2.5 Deliverables:

- Grammar **of the language defined in 2.1.**
- **Parser:** Python code that takes tokens and builds an AST.

Milestone 3: Semantic Analysis

The semantic analyzer ensures that the program has meaningful, logical structure. It checks for type correctness, variable declarations, scope resolution, and other semantic errors. It doesn't change the syntax tree, but it ensures the operations make sense.

3.1 Tasks:

- **Variable Type Checking:** Ensure that variables are assigned values of compatible types (e.g., no assigning strings to integer variables).
- **Scope Checking:** Ensure variables are used only after they are declared.
- **Expression Checking:** Ensure that operations like x + 2 are valid based on the variable types (e.g., adding an integer and a string should result in an error).

  Implement the symbol table and perform checks on variable declarations and types. Create error messages for invalid operations or undeclared variables. Test with programs that contain semantic errors.

3.2 Example:

- If a program contains x = "hello";, the semantic analyzer should detect that "hello" is a string and raise an error if x is not declared as a string.

- Check that all variables used in expressions are defined.

3.3 Tools to use:

- **Symbol Table**: A data structure to keep track of variable types and scopes.
- **Manual Checking**: Implement checks based on the AST generated in the previous step.

3.4 Deliverables:

- **Semantic Analyzer:** Python code that performs type checking and scope resolution.
- **Test Cases:** A set of programs for testing the compiler.